IMPORT LIBS, AI, DATASET, AND PLOTTING

In [304…

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import random_split
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
```

DEFINE GLOBAL CONSTANTS AND VARS

In [305…

```
BATCH_SIZE = 64
TRAIN_SIZE = 45000
VAL_SIZE = 5000
LOG_INTERVAL = 75

NOISE_FACTOR = 0.7
IMGS_BY_ROW = 8
EPOCHS = 5

SEED = 83
RAND_GEN = torch.Generator().manual_seed(SEED)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

DEFINE UTILITY FUNCTIONS

In [306…

```
def get_normalize_stats(loader):
    mean = torch.zeros(3)
    std = torch.zeros(3)
    n_samples = 0

    for batch, _ in loader:
        batch_size = batch.size(0)
        batch_mean = batch.mean(dim=(0, 2, 3))
        batch_std  = batch.std(dim=(0, 2, 3))

        mean += batch_mean * batch_size
```

```
        std  += batch_std * batch_size
        n_samples += batch_size

    mean = np.round(mean / n_samples, 4)
    std  = np.round(std / n_samples, 4)
    return mean, std

def denormalize(img, mean, std):
    mean = mean[:, None, None]
    std = std[:, None, None]
    return img * std + mean

def add_noise(tensor):
    noisy_img = tensor + NOISE_FACTOR * torch.rand(*tensor.shape)
    noisy_img = torch.clamp(noisy_img, -2., 2.)
    return noisy_img
```

DEFINE SHOW SAMPLE IMG AND NOISE FUNCTION

In [307…

```
def show_noise(images, row_len, model=None):

    if len(images) < row_len:
        print("ERROR: Too few images for row")

    num_of_rows = 2 if model is None else 3
    figsize = (18, 6) if model is None else (18, 9)
    fig = plt.figure(figsize=figsize)

    for i in range(row_len):
        img = images[i]
        clean_img = denormalize(img, _mean, _std)
        clean_img = clean_img.permute(1, 2, 0)

        clean_ax = plt.subplot(num_of_rows, row_len, i+1)
        clean_ax.imshow(clean_img)
        clean_ax.axis('off')

        noisy_w_norm = add_noise(img)
        noisy_img = denormalize(noisy_w_norm, _mean, _std)
        noisy_img = noisy_img.permute(1, 2, 0)
```

```
        noisy_ax = plt.subplot(num_of_rows, row_len, i+1+row_len)
        noisy_ax.imshow(noisy_img)
        noisy_ax.axis('off')

        if model is not None:
            output = model(noisy_w_norm)
            output = output.detach()
            output = denormalize(output, _mean, _std)
            output = output.permute(1, 2, 0)

            out_ax = plt.subplot(3, 8, i+1+row_len*2)
            out_ax.imshow(output)
            out_ax.axis('off')

    if model is not None:
        fig.text(0.5, 0.87, 'Raw', ha="center", fontsize=24)
        fig.text(0.5, 0.60, 'With Noise', ha="center", fontsize=24)
        fig.text(0.5, 0.32, 'From Model', ha="center", fontsize=24)
    else:
        fig.text(0.5, 0.85, 'Raw', ha="center", fontsize=24)
        fig.text(0.5, 0.44, 'With Noise', ha="center", fontsize=24)

    plt.show()
```

DEFINE THE AUTOENCODER

In [308…

```
class DenoisingAutoencoder(nn.Module):
    def __init__(self):
        super(DenoisingAutoencoder, self).__init__()
        self.conv1 = nn.Conv2d(3, 8, 3, padding=1)
        self.conv2 = nn.Conv2d(8, 16, 3, padding=1)
        self.pool = nn.MaxPool2d(2, stride=2)

        self.t_conv1 = nn.ConvTranspose2d(16, 8, 2, stride=2)
        self.conv_out = nn.Conv2d(8, 3, 3, padding=1)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = self.pool(x)
```

```
        x = F.relu(self.t_conv1(x))
        x = F.tanh(self.conv_out(x))

        return x
```

DEFINE TRAINING FUNCTION

In [309…

```
def train_model(model, optimizer, criterion, train_loader, val_loader):
    train_loss = []
    val_loss = []
    for epoch in range(1, EPOCHS+1):
        for i, (imgs, _) in enumerate(train_loader):

            noisy_imgs = add_noise(imgs).to(device)
            imgs = imgs.to(device)

            optimizer.zero_grad()
            outputs = model(noisy_imgs)

            loss = criterion(outputs, imgs)
            loss.backward()
            optimizer.step()

            train_loss.append(loss.item())
            if i % LOG_INTERVAL == 0:
                print(f"EPOCH {str(epoch).ljust(2)}| ITER {str(i).ljust(5)} | TRAIN LOSS {loss.item()}")

        model.eval()
        with torch.no_grad():
            for i, (imgs, _) in enumerate(val_loader):

                noisy_imgs = add_noise(imgs).to(device)
                imgs = imgs.to(device)

                outputs = model(noisy_imgs)
                loss = criterion(outputs, imgs)

                val_loss.append(loss.item())
                if i % LOG_INTERVAL == 0:
                    print(f"EPOCH {str(epoch).ljust(2)}| ITER {str(i).ljust(5)} | VAL   LOSS {loss.item()}")
```

```
    print("NOTICE: TRAINING IS DONE")
    return train_loss, val_loss
```

FETCH, SPLIT, AND NORMALIZE DATA

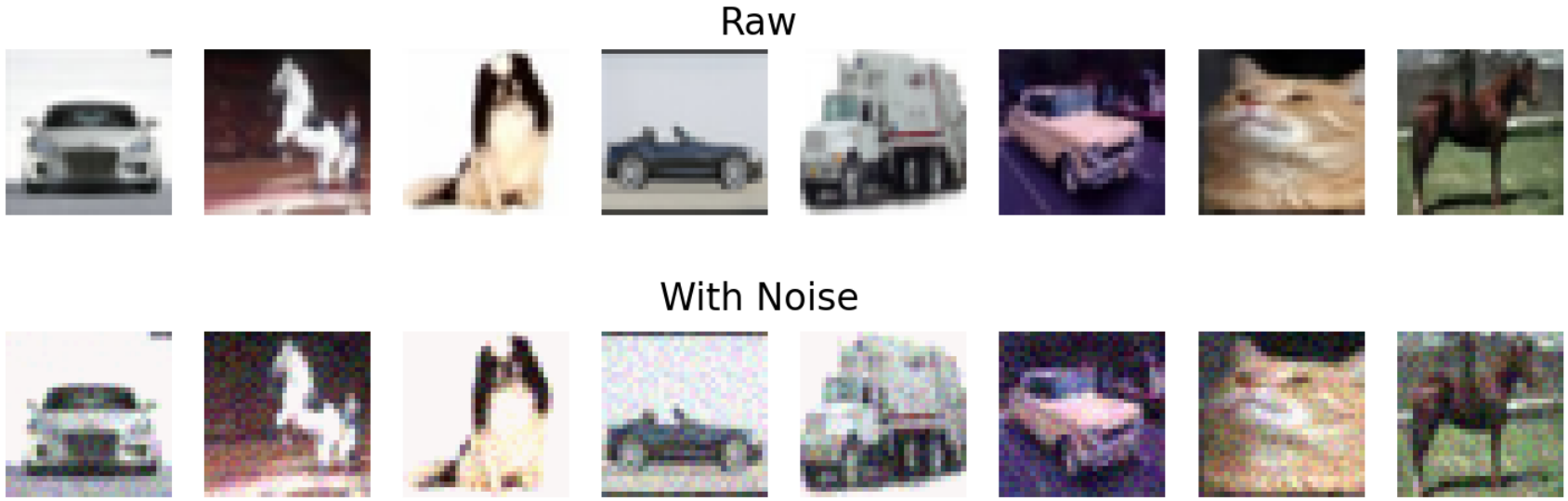
In [310…

```
rawset = datasets.CIFAR10('/content/data', train=True, download=True, transform=transforms.ToTensor())
raw_loader = torch.utils.data.DataLoader(dataset=rawset, batch_size=BATCH_SIZE)

_mean, _std = get_normalize_stats(raw_loader)
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(_mean, _std)
])

# Rawset was just for getting normalize stats
# CIFAR10 will be ready in the files and not download again
dataset = datasets.CIFAR10('/content/data', train=True, download=True, transform=transform)
trainset, valset = random_split(
    dataset=dataset,
    lengths = [TRAIN_SIZE, VAL_SIZE],
    generator = RAND_GEN
)

train_loader = torch.utils.data.DataLoader(dataset = trainset, batch_size = BATCH_SIZE)
val_loader = torch.utils.data.DataLoader(dataset = valset, batch_size = BATCH_SIZE)

testset = datasets.CIFAR10('/content/data', train=False, download=True, transform=transform)
test_loader = torch.utils.data.DataLoader(dataset = testset, batch_size = BATCH_SIZE)
```

SAMPLES IMAGES OF TRAINING SET BEFORE/AFTER NOISE, THE NOISY IMAGES WILL BE SENT TO THE MODEL

In [311…

```
sample_imgs, _ = next(iter(train_loader))
show_noise(sample_imgs, IMGS_BY_ROW)
```




TRAIN MODEL

In [312…

```
model = DenoisingAutoencoder().to(device)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
train_loss, val_loss = train_model(model, optimizer, criterion, train_loader, val_loader)
```

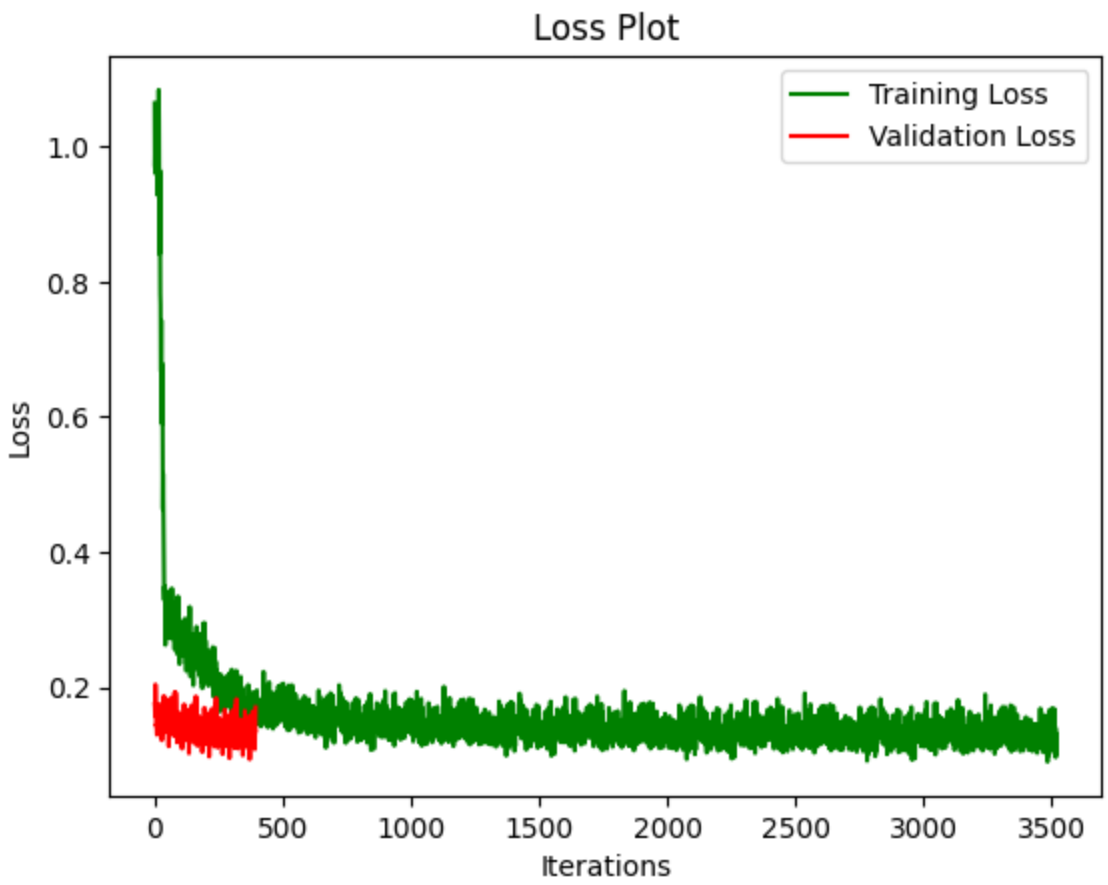
```
EPOCH 1 | ITER 0     | TRAIN LOSS 1.064441442489624
EPOCH 1 | ITER 75    | TRAIN LOSS 0.2917638421058655
EPOCH 1 | ITER 150   | TRAIN LOSS 0.23995904624462128
EPOCH 1 | ITER 225   | TRAIN LOSS 0.24662213027477264
EPOCH 1 | ITER 300   | TRAIN LOSS 0.1856202334165573
EPOCH 1 | ITER 375   | TRAIN LOSS 0.16929040849208832
EPOCH 1 | ITER 450   | TRAIN LOSS 0.16079871356487274
EPOCH 1 | ITER 525   | TRAIN LOSS 0.1440548151731491
EPOCH 1 | ITER 600   | TRAIN LOSS 0.14794935286045074
EPOCH 1 | ITER 675   | TRAIN LOSS 0.18696379661560059
EPOCH 1 | ITER 0     | VAL   LOSS 0.17484350502490997
EPOCH 1 | ITER 75    | VAL   LOSS 0.1279447227716446
EPOCH 2 | ITER 0     | TRAIN LOSS 0.1694059818983078
EPOCH 2 | ITER 75    | TRAIN LOSS 0.15669532120227814
EPOCH 2 | ITER 150   | TRAIN LOSS 0.14190979301929474
EPOCH 2 | ITER 225   | TRAIN LOSS 0.15493595600128174
EPOCH 2 | ITER 300   | TRAIN LOSS 0.14654962718486786
EPOCH 2 | ITER 375   | TRAIN LOSS 0.142587348818779
EPOCH 2 | ITER 450   | TRAIN LOSS 0.1364157646894455
EPOCH 2 | ITER 525   | TRAIN LOSS 0.12479502707719803
EPOCH 2 | ITER 600   | TRAIN LOSS 0.1321849673986435
EPOCH 2 | ITER 675   | TRAIN LOSS 0.1749630719423294
EPOCH 2 | ITER 0     | VAL   LOSS 0.16265803575515747
EPOCH 2 | ITER 75    | VAL   LOSS 0.11721280217170715
EPOCH 3 | ITER 0     | TRAIN LOSS 0.1564863920211792
EPOCH 3 | ITER 75    | TRAIN LOSS 0.14724431931972504
EPOCH 3 | ITER 150   | TRAIN LOSS 0.13293124735355377
EPOCH 3 | ITER 225   | TRAIN LOSS 0.1461694985628128
EPOCH 3 | ITER 300   | TRAIN LOSS 0.13895374536514282
EPOCH 3 | ITER 375   | TRAIN LOSS 0.13612867891788483
EPOCH 3 | ITER 450   | TRAIN LOSS 0.1301482915878296
EPOCH 3 | ITER 525   | TRAIN LOSS 0.11886313557624817
EPOCH 3 | ITER 600   | TRAIN LOSS 0.1269906908273697
EPOCH 3 | ITER 675   | TRAIN LOSS 0.17011289298534393
EPOCH 3 | ITER 0     | VAL   LOSS 0.1575399488210678
EPOCH 3 | ITER 75    | VAL   LOSS 0.11219900846481323
EPOCH 4 | ITER 0     | TRAIN LOSS 0.15099678933620453
EPOCH 4 | ITER 75    | TRAIN LOSS 0.14289891719818115
EPOCH 4 | ITER 150   | TRAIN LOSS 0.12904947996139526
EPOCH 4 | ITER 225   | TRAIN LOSS 0.14186261594295502
EPOCH 4 | ITER 300   | TRAIN LOSS 0.1350852996110916
EPOCH 4 | ITER 375   | TRAIN LOSS 0.1328163594007492
```

```
EPOCH 4 | ITER 450   | TRAIN LOSS 0.12718389928340912
EPOCH 4 | ITER 525   | TRAIN LOSS 0.11568247526884079
EPOCH 4 | ITER 600   | TRAIN LOSS 0.12453000992536545
EPOCH 4 | ITER 675   | TRAIN LOSS 0.16752545535564423
EPOCH 4 | ITER 0     | VAL   LOSS 0.15497374534606934
EPOCH 4 | ITER 75    | VAL   LOSS 0.10971704870462418
EPOCH 5 | ITER 0     | TRAIN LOSS 0.1481744647026062
EPOCH 5 | ITER 75    | TRAIN LOSS 0.14058157801628113
EPOCH 5 | ITER 150   | TRAIN LOSS 0.12661144137382507
EPOCH 5 | ITER 225   | TRAIN LOSS 0.13934941589832306
EPOCH 5 | ITER 300   | TRAIN LOSS 0.13309822976589203
EPOCH 5 | ITER 375   | TRAIN LOSS 0.1309434026479721
EPOCH 5 | ITER 450   | TRAIN LOSS 0.1254589855670929
EPOCH 5 | ITER 525   | TRAIN LOSS 0.11406826972961426
EPOCH 5 | ITER 600   | TRAIN LOSS 0.12314498424530029
EPOCH 5 | ITER 675   | TRAIN LOSS 0.16614839434623718
EPOCH 5 | ITER 0     | VAL   LOSS 0.153261199593544
EPOCH 5 | ITER 75    | VAL   LOSS 0.10832182317972183
NOTICE: TRAINING IS DONE
```

EVALUATE MODEL TRAINING

In [313…

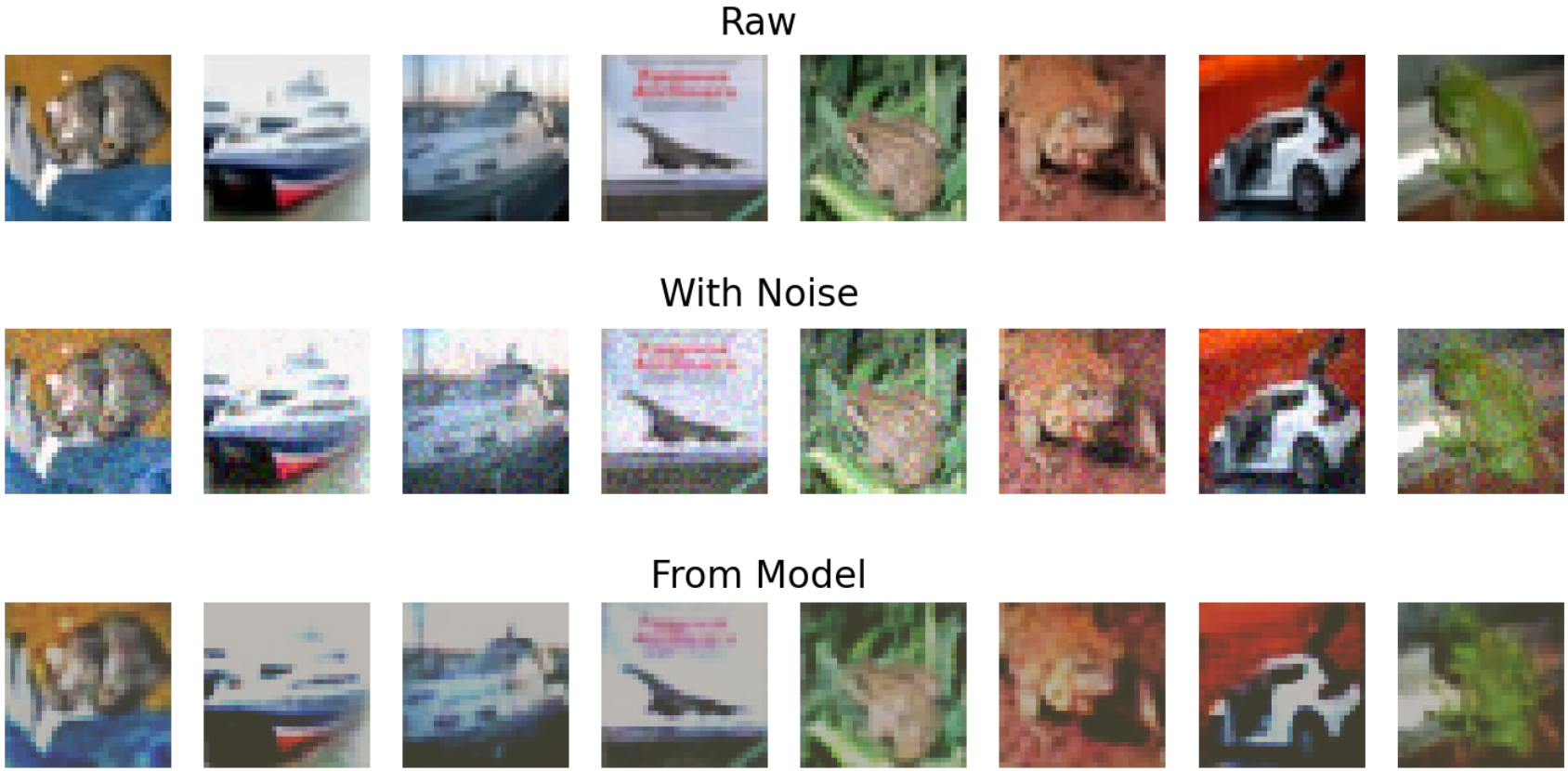
```
plt.figure()
plt.plot(train_loss, color='green', label='Training Loss')
plt.plot(val_loss, color='red', label='Validation Loss')
plt.title('Loss Plot')
plt.xlabel('Iterations')
plt.ylabel('Loss')
_ = plt.legend()
```




EVALUATE MODEL OUTPUT

In [314…

```
test_imgs, _ = next(iter(test_loader))
show_noise(test_imgs, IMGS_BY_ROW, model)
```


Raw
With Noise
From Model